Together Fund — Intern Assignment

Part 1: Ideation Document

Kushagra | July 2026

Before jumping to solutions, I spent time thinking about where Together Fund actually loses hours. The five pain points below came from reading how operator-led VC firms work at scale — deal intake, diligence, portfolio monitoring, meeting memory, LP reporting — and mapping each to a concrete build. I've also added a sixth that feels underutilised given Together's 150+ operator network. Each idea is scoped to be buildable solo in 2 days, not a six-month product.

The Six Ideas

Issue 1 — Deal Flow Intake Agent

Problem: Pitches arrive via email, LinkedIn, and Typeform with no unified view. Partners spend 30 minutes per pitch just reading and categorising before they can even decide if it's worth a call.

What it does: Ingests pitch decks (PDF), websites, and LinkedIn profiles. Extracts founder background, market, business model, and traction signals. Scores each deal against Together's investment thesis (B2B SaaS, AI-native, Seed/Series A). Outputs a structured one-pager with a fit score.

Architecture: FastAPI + Claude API. PDF extraction via pdfplumber, website scraping via httpx. Structured extraction into a Pydantic schema, stored in SQLite with a React pipeline dashboard.

Non-trivial elements: Multi-step agentic pipeline (scrape → extract → score → store), structured output with schema validation, tool use combining web fetch and PDF parse.

Issue 2 — Due Diligence Research Agent

Problem: A partner spends 3–4 hours per deal manually researching competitors, market size, founder history, news mentions, and similar funded companies. Across 8 deals a week, that's a full workday.

What it does: Given a company name and description, the agent runs a multi-step research loop — web searches for competitors, funding rounds, market reports, and founder credibility signals. Synthesises a structured 2-page DD memo with citations.

Architecture: Claude with web search tool use in an agentic loop. Each search step is logged visibly. Output is a formatted memo. FastAPI backend, Streamlit frontend showing live research steps as they happen.

Non-trivial elements: True agentic loop with tool use, visible reasoning trace, multi-source synthesis, structured Pydantic output with source attribution.

Issue 3 — Meeting Memory & CRM Agent

Problem: 8+ founder calls per week. Notes are scattered or absent. Partners forget context between meetings and have no searchable record of what was promised or discussed.

What it does: Takes a meeting transcript (pasted or uploaded). Extracts action items, founder asks, partner commitments, key metrics, and next steps. Stores in a searchable vector DB. Before the next meeting, retrieves relevant prior context automatically.

Architecture: FastAPI + Claude for structured extraction. ChromaDB for vector storage and retrieval. React frontend for input and search. Semantic retrieval keyed on company name and founder name.

Non-trivial elements: Structured extraction into a typed schema, vector memory with retrieval, multi-meeting context threading across sessions.

Issue 4 — Portfolio Health Monitor

Problem: Partners learn about portfolio problems when founders call in crisis — layoffs announced, runway at three weeks, co-founder gone. With 36+ companies, there's no systematic early-warning layer.

What it does: Monitors public signals per portfolio company — LinkedIn headcount trends, job postings, news mentions, GitHub activity. Flags anomalies weekly. Partners open a dashboard Monday morning and immediately know which companies need attention.

Architecture: Scheduled Python jobs (APScheduler) for signal collection. Groq/Llama for anomaly interpretation. SQLite for trend storage. Streamlit dashboard with red/yellow/green health status and drill-down reasoning per company.

Non-trivial elements: Multi-source monitoring, week-over-week trend detection, LLM-powered anomaly classification with structured HealthSummary output.

Issue 5 — LP Report Generator

Problem: Quarterly LP reports are assembled manually from portfolio updates, news, and fund metrics. It takes days and the output quality varies depending on who writes it that quarter.

What it does: Takes structured portfolio data (funding rounds, revenue milestones, headcount, news) as input. Generates a professional LP update memo — fund performance narrative, portfolio highlights, market context, forward outlook — in Together Fund's voice.

Architecture: Claude with a detailed system prompt calibrated to Together's communication style. Input via JSON/CSV upload. Output as formatted PDF. FastAPI backend.

Non-trivial elements: Long-form structured generation, multi-section document with consistent voice, data grounding to prevent hallucination.

Issue 6 — Founder-Operator Match Engine

Problem: Together's 150+ operator network (SwarmSpace) is underutilised. Partners don't have an easy way to know which operator to connect to which portfolio founder for a specific challenge.

What it does: Ingests operator profiles (expertise, industry, availability) and portfolio company needs. When a founder states a challenge — 'need someone who scaled a SaaS sales team in APAC' — it finds the top 3 matching operators with reasoning.

Architecture: ChromaDB for operator profiles embedded by expertise. Claude for need extraction from natural language. Semantic retrieval with Claude re-ranking for final matches. Simple Streamlit UI.

Non-trivial elements: RAG-based matching, semantic retrieval over structured profiles, Claude re-ranking with explicit match reasoning.

Prioritised Shortlist — Why These Two First

Out of the six ideas, I built Tool 1 (DD Research Agent) and Tool 2 (Portfolio Health Monitor). Here's the reasoning behind that choice.

Tool 1 — Due Diligence Research Agent (Issue 2)

This one comes first because it's the most self-contained. Input → agent loop → structured output, no database setup, no scheduler, no seed data complexity. It's also the most immediately legible to a non-technical partner watching a demo — they can see the agent thinking in real time and understand exactly what it's doing. The agentic loop with live streaming reasoning is technically the most interesting artifact to evaluate.

— Daily-use value: every incoming pitch triggers a research run. Direct time saving of 3–4 hours per deal.
— Technical depth: true agentic loop, tool use, structured output, SSE streaming, source attribution.
— Buildable in one day: five files, no external dependencies beyond a search API and an LLM.

Tool 2 — Portfolio Health Monitor (Issue 4)

This comes second because it needs slightly more setup — SQLite, APScheduler, seed data — but the payoff is a visually rich dashboard that tells an immediate story. The core insight is that the value isn't in the scraping, it's in the interpretation layer on top. Any script can pull a headcount number. What matters is reading headcount + news + hiring together and surfacing a concrete partner action.

— Daily-use value: partners open this on Monday morning before their weekly portfolio review call.
— Technical depth: multi-source signal fusion, week-over-week trend detection, structured LLM health summaries.
— Non-obvious build: the conditional agent branching (fallback search if initial returns < 2 results) and the 6-hour cache guard are what separate this from a simple scraper.

Why not the others first?

Issue 1 (Deal Flow Intake) overlaps heavily with Tool 1 — both process inbound pitches. Building both would be redundant for a demo. Issue 3 (Meeting Memory) is compelling but requires a real transcript to demo well, which adds friction. Issues 5 and 6 are high-value but lower on daily-use frequency — LP reports are quarterly, and the operator match engine requires a populated operator database that takes time to seed properly.

Honest Tradeoffs & Risks

DD Research Agent

— **Search rate limits:** DuckDuckGo blocks repeated queries at the IP level during live demos. Switched to Tavily Search API which is purpose-built for LLM agents — structured results, no blocking, 1000 free searches/month.
— **Hallucination risk:** Claude can confuse companies with similar names. Every claim in the memo is tagged to its source URL. If fewer than 3 sources are found, the memo header is flagged as low-confidence.
— **Thin public footprint:** very early-stage startups with no web presence produce sparse memos. The agent surfaces this explicitly rather than generating plausible-sounding content.

Portfolio Health Monitor

— **LinkedIn scraping:** LinkedIn blocks aggressive scraping and it violates ToS at scale. For the demo, headcount history is realistic synthetic data seeded per company. In production this would be Proxycurl or founder-reported quarterly updates.
— **Public signals only:** stealth-mode companies with minimal web presence will always return low-confidence verdicts. The UI makes this visible — each card shows exactly which signal sources returned data and which returned nothing.
— **duckduckgo-search v6+ breaks on Windows:** v6 introduced pyreqwest-impersonate which requires Rust/Cargo to compile. Pinned to 5.3.1b1 — the last version that installs cleanly via pip on Windows without a Rust toolchain.

Both tools are independently runnable. Each has its own subfolder, requirements file, and README. The shared virtual environment works for both — the core dependency overlap (groq, pydantic, python-dotenv,

duckduckgo-search) is identical and conflict-free.